

Oracle SQL Developer Practice Guide

96 Structured SQL Exercises covering 16 core database topics (6 questions per topic). Grounded in the AUCA *INSY 8311 PL/SQL* course database schema.

DATABASE SCHEMA QUICK REFERENCE

Use these exact table and column names when writing your SQL queries in Oracle SQL Developer:

Table Name	Columns & Data Types
STUDENTS	student_id, first_name, last_name, email, phone, date_of_birth, enrollment_date, gpa, status
COURSES	course_id, course_name, course_code, credits, instructor, department, max_students, semester
ENROLLMENTS	enrollment_id, student_id, course_id, enrollment_date, grade, score, status
EMPLOYEES	employee_id, first_name, last_name, email, phone, hire_date, job_title, salary, commission_pct, department, manager_id
DEPARTMENTS	department_id, department_name, manager_id, location, budget

1. Select Basics (All & Specific Columns)

- Q1.1** Write a SQL query in Oracle SQL Developer to retrieve all columns and rows from the STUDENTS table.
- Q1.2** Write a query to display only the first_name, last_name, and gpa of every student in the STUDENTS table.
- Q1.3** Write a query to retrieve the course_name, course_code, and credits from the COURSES table.
- Q1.4** Write a query to select all columns from the EMPLOYEES table to inspect complete employee records.
- Q1.5** Write a query to display the employee_id, first_name, job_title, and department from the EMPLOYEES table.
- Q1.6** Write a query to display enrollment_id, student_id, course_id, and grade from the ENROLLMENTS table.

2. Alias, Calculated Columns & String Concatenations

- Q2.1** Write a query using the concatenation operator (||) to combine first_name and last_name into a single column aliased as full_name for all employees.
- Q2.2** Retrieve first_name, last_name, and create a calculated column annual_salary (salary * 12) from the EMPLOYEES table.
- Q2.3** Write a query on STUDENTS to create a formatted contact string aliased as student_contact in the format 'Name: ' || first_name || ' (' || email || ')'.
- Q2.4** Write a query displaying employee_id, job_title, and a calculated column bonus_amount representing 10% of monthly salary (salary * 0.10).
- Q2.5** Write a query on COURSES to display course_code and course_name concatenated as course_full_info (e.g., 'CS101 - Introduction to Programming').
- Q2.6** Write a query to display student_id, gpa, and calculated expected_gpa_increase (gpa + 0.20) aliased as target_gpa from STUDENTS.

3. Handling Null Values (IS NULL, NOT NULL, NVL)

- Q3.1** Write a query to list all employees who do not receive a commission (commission_pct IS NULL).
- Q3.2** Write a query to display all employees who have an assigned manager (manager_id IS NOT NULL).
- Q3.3** Use the NVL function to display commission_pct for employees, substituting 0 whenever a NULL value is encountered.
- Q3.4** Query the ENROLLMENTS table to find all course enrollment records where grade IS NULL.
- Q3.5** Write a query on DEPARTMENTS to find all departments where manager_id IS NULL.
- Q3.6** Use NVL to display grade in ENROLLMENTS, substituting 'Ungraded' for any NULL grade records.

4. WHERE Clause Basics

- Q4.1** Write a query to select all student records from STUDENTS where status is 'Active'.
- Q4.2** Write a query to list all courses in COURSES offered by the 'Computer Science' department.
- Q4.3** Write a query to select all employees from EMPLOYEES assigned to the 'Business' department.
- Q4.4** Write a query on ENROLLMENTS to list all enrollments where the numerical score is greater than 90.0.
- Q4.5** Write a query to select all departments located in 'Building A - Floor 3' from the DEPARTMENTS table.
- Q4.6** Write a query to display all students who enrolled on the exact date '2023-09-01'.

5. Comparison Operators (=, <>, !=, >, <, >=, <=)

- Q5.1** Write a query to find all students with a gpa greater than or equal to 3.50 (gpa >= 3.5).
- Q5.2** Write a query to select all courses from COURSES where credits are not equal to 3 (credits <> 3).
- Q5.3** Write a query to list all employees whose salary is strictly less than 60000 (salary < 60000).
- Q5.4** Write a query to list courses where max_students capacity is strictly greater than 30.
- Q5.5** Write a query on ENROLLMENTS to select all records where score is less than or equal to 75.00 (score <= 75.0).
- Q5.6** Write a query to find departments with a budget strictly greater than 500000 (budget > 500000).

6. String & Pattern Matching (LIKE)

- Q6.1** Write a query using LIKE to find all students whose first_name starts with the letter 'A'.
- Q6.2** Write a query to find all employees whose job_title contains the word 'Developer' (job_title LIKE '%Developer%').
- Q6.3** Write a query to select all courses whose course_code ends with '01'.
- Q6.4** Write a query to find students whose email ends with '@email.com' and whose last_name starts with 'S'.
- Q6.5** Write a query to find courses whose course_name contains the word 'Programming'.
- Q6.6** Write a query on EMPLOYEES to select all employees whose first_name has 'a' as its second letter (e.g., LIKE '_a%').

7. Between and IN Operators

- Q7.1** Write a query to retrieve all students whose gpa falls between 3.0 and 3.8 inclusive (gpa BETWEEN 3.0 AND 3.8).
- Q7.2** Write a query using IN to select employees working in either 'Computer Science', 'Mathematics', or 'Engineering'.
- Q7.3** Write a query using IN to find all courses that offer either 3 or 4 credits.
- Q7.4** Write a query on ENROLLMENTS to list all student scores falling between 80.00 and 90.00.
- Q7.5** Write a query on EMPLOYEES to list all staff hired between '2019-01-01' and '2021-12-31'.
- Q7.6** Write a query using NOT IN to display courses that are NOT in 'Psychology' or 'English'.

8. Logical Operators (AND, OR, NOT)

- Q8.1** Write a query to select students who have a gpa ≥ 3.5 AND a status of 'Active'.
- Q8.2** Write a query to select employees who work in the 'Business' department OR have a salary > 80000 .
- Q8.3** Write a query to find courses where department is 'Computer Science' AND credits = 4.
- Q8.4** Write a query using NOT to select students whose gpa is NOT less than 3.0.
- Q8.5** Write a query on EMPLOYEES to find employees in 'Computer Science' with salary > 70000 AND manager_id = 1.
- Q8.6** Write a query to find courses where department is 'Mathematics' OR department is 'Business' AND credits ≥ 3 .

9. Date Comparison & EXTRACT Keyword

- Q9.1** Write a query to find all students who enrolled on or after January 1, 2024 (enrollment_date $\geq$ DATE '2024-01-01').
- Q9.2** Write a query using EXTRACT(YEAR FROM hire_date) to display the hiring year for each employee in EMPLOYEES.
- Q9.3** Write a query using EXTRACT(MONTH FROM date_of_birth) to retrieve the birth month of all students.
- Q9.4** Write a query to list all employees hired in the year 2020 using EXTRACT(YEAR FROM hire_date) = 2020.
- Q9.5** Write a query to find students born before the year 2004 (date_of_birth $<$ DATE '2004-01-01').
- Q9.6** Write a query using EXTRACT(DAY FROM enrollment_date) to show the day of the month students enrolled.

10. ORDER BY Basics & NULLS Handling

- Q10.1** Write a query to list all students ordered by gpa from highest to lowest (ORDER BY gpa DESC).
- Q10.2** Write a query to select all employees ordered by department ascending, and then by salary descending within each department.
- Q10.3** Write a query to list employees sorted by commission_pct in ascending order, placing NULL values at the bottom (NULLS LAST).
- Q10.4** Write a query to order all courses by max_students capacity descending.
- Q10.5** Write a query to display ENROLLMENTS sorted by score ASC, placing NULL scores at the top (NULLS FIRST).
- Q10.6** Write a query to list all employees sorted alphabetically by last_name, then by first_name.

11. Single Row Functions (UPPER, LOWER, LENGTH, TO_CHAR, ROUND, TRUNC, ADD_MONTHS)

- Q11.1** Write a query displaying UPPER(first_name) and LOWER(email) for all students in the STUDENTS table.
- Q11.2** Write a query using LENGTH(course_name) to display the number of characters in each course name from COURSES.
- Q11.3** Write a query using TO_CHAR(hire_date, 'YYYY-MM-DD') to display employee hire dates in a formatted string.
- Q11.4** Write a query showing ROUND(salary/12, 2) alongside TRUNC(salary/12, 2) for employees.
- Q11.5** Write a query using ADD_MONTHS(hire_date, 6) to calculate a 6-month probation review date for each employee.
- Q11.6** Write a query displaying UPPER(job_title) and LENGTH(last_name) for all employees.

12. Conditional Logic with CASE

- Q12.1** Write a query on STUDENTS using a CASE statement to categorize students into 'Honors' (gpa >= 3.7), 'Good Standing' (gpa BETWEEN 3.0 AND 3.69), and 'Needs Improvement' (gpa < 3.0).
- Q12.2** Write a CASE statement on EMPLOYEES to label salaries as 'High' (> 80,000), 'Medium' (60,000 - 80,000), or 'Entry' (< 60,000).
- Q12.3** Write a query on ENROLLMENTS using CASE to classify numeric score as 'Pass' (score >= 75.0) or 'Fail' (score < 75.0).
- Q12.4** Write a CASE statement on COURSES to label 4-credit courses as 'Intensive' and 3-credit courses as 'Standard'.
- Q12.5** Write a CASE statement on EMPLOYEES to assign a bonus percentage: 15% for 'Department Head', 10% for 'Senior Developer', 5% for all others.
- Q12.6** Write a CASE statement on STUDENTS to label students enrolled in 2023 as 'Senior Cohort' and 2024 as 'Junior Cohort'.

13. Aggregate Functions (COUNT, AVG, SUM, MIN, MAX)

- Q13.1** Write a query using COUNT(*) to return the total number of students in the STUDENTS table.
- Q13.2** Write a query calculating AVG(gpa), MIN(gpa), and MAX(gpa) for all active students.
- Q13.3** Write a query using SUM(budget) to calculate the overall budget across all entries in DEPARTMENTS.
- Q13.4** Write a query to find the AVG(salary), MIN(salary), and MAX(salary) across all staff in the EMPLOYEES table.
- Q13.5** Write a query using COUNT(DISTINCT department) to count how many unique departments exist in EMPLOYEES.
- Q13.6** Write a query to calculate SUM(salary) to find the total payroll spent on employees in EMPLOYEES.

14. Grouping & HAVING Clause

- Q14.1** Write a query using GROUP BY department to count the total number of employees in each department.
- Q14.2** Write a query calculating average salary per department and use HAVING AVG(salary) > 65000 to display only departments exceeding that threshold.
- Q14.3** Write a query on ENROLLMENTS using GROUP BY course_id to show how many students are enrolled in each course.
- Q14.4** Write a query on COURSES grouped by department using HAVING COUNT(*) > 2 to show departments offering more than two courses.
- Q14.5** Write a query on ENROLLMENTS grouped by student_id calculating AVG(score), filtered with HAVING AVG(score) >= 85.0.
- Q14.6** Write a query on EMPLOYEES grouped by job_title to show the count and total salary per job title.

15. Joins (Inner Join, Left Join, Right Join)

- Q15.1** Write an INNER JOIN query between STUDENTS and ENROLLMENTS on student_id to display student names alongside their enrolled course_id and grade.
- Q15.2** Write a 3-table INNER JOIN linking STUDENTS, ENROLLMENTS, and COURSES to display first_name, last_name, course_name, and score.
- Q15.3** Write a LEFT JOIN between DEPARTMENTS and EMPLOYEES on department_name / department to list all departments regardless of whether employees are assigned.
- Q15.4** Write a LEFT JOIN from COURSES to ENROLLMENTS on course_id to ensure courses with zero active enrollments are still returned.
- Q15.5** Write a RIGHT JOIN between EMPLOYEES and DEPARTMENTS to list all departments and their associated employee details.
- Q15.6** Write a Self-JOIN on EMPLOYEES (joining EMPLOYEES e with EMPLOYEES m on e.manager_id = m.employee_id) to display employee name and manager name.

16. Subqueries

- Q16.1** Write a query with a single-row subquery to select employees whose salary is greater than the overall average salary (`salary > (SELECT AVG(salary) FROM employees)`).
- Q16.2** Write a query using a subquery with IN to find all student names enrolled in the 'PL/SQL Programming' course (CS202).
- Q16.3** Write a subquery to list all students whose gpa is strictly higher than Bob Smith's GPA.
- Q16.4** Write a query to find all departments in DEPARTMENTS whose budget exceeds the average budget across all departments.
- Q16.5** Write a subquery using NOT IN to find students who have never enrolled in any course in the ENROLLMENTS table.
- Q16.6** Write a correlated subquery or nested subquery to find the employee with the maximum salary in each department.